



COM3021 – Data Science at Scale

Data Science at Scale Review

DR YANDA MENG

Reliability

Reliability

- The application performs the function that the user expected.
- It can tolerate the user making mistakes or using the software in unexpected ways.
- Its performance is good enough for the required use case, under the expected load and data volume.
- The system prevents any unauthorized access and abuse.

Resilient systems

Fault-tolerance and resilience

- Fault

When one component of the system works in an unexpected way

Hardware

Software

- Failure

When the entire system stops providing the service

Scalability

Scalability

- A system's ability to cope with increased load
 - How do we define load?*
 - Defined in terms of *load parameters*
 - Requests per second for web servers*
 - Read/write ratio for databases*
 - Number of simultaneous players in an online game*
 - Average case vs outliers
-

Maintainability

Maintainability

- The maintainability of a system is the overall cost to maintain a system operational and updated
 - Operability*
 - How easy it is for the operation team to keep it running

Simplicity

- How easy it is for new people working on the system to understand it

Evolvability

- How easy it is for engineers to make changes and update to the system (e.g., for new requirements, for increased load)
-

Consistency

The CAP theorem

Distributed systems? Choose two:

- Consistency
Every read receives the most recent write or an error.
- Availability
Every request receives a (non-error) response.
- Partition tolerance

The system continues to operate even if parts of the network become disconnected

A.C.I.D. properties

- Atomicity

Each statement in a transaction (to read, write, update or delete data) is treated as a single unit.

- Consistency

Ensures that transactions only make changes to tables in predefined, predictable ways

- Isolation

When multiple users are reading and writing from the same table all at once, isolation of their transactions ensures that the concurrent transactions don't interfere with or affect one another

- Durability

- Ensures that changes to your data made by successfully executed transactions will be saved, even in the event of system failure.
-

Eventual consistency

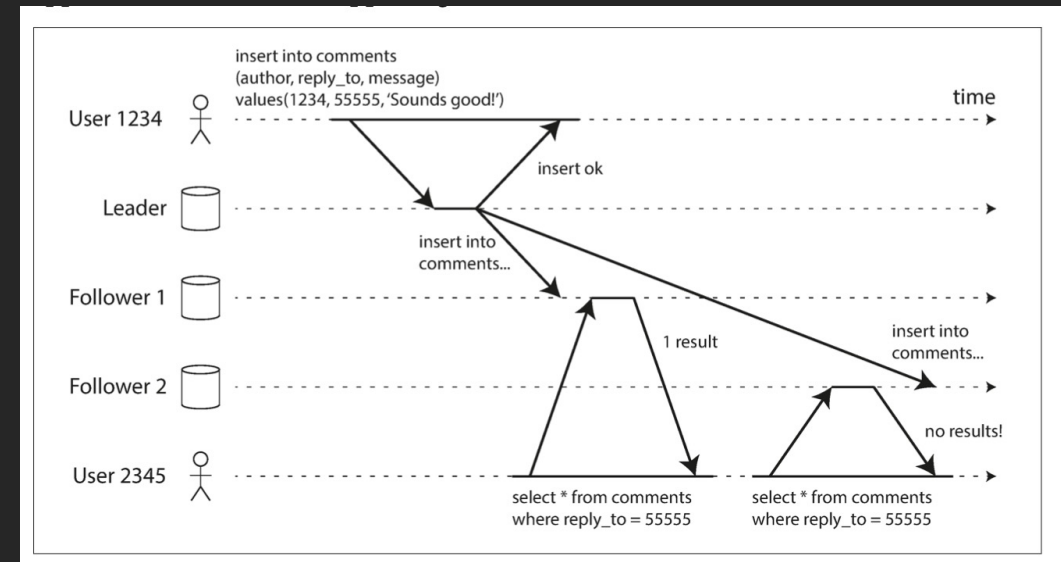
- At a given point in time, some data nodes in a distributed system will have an outdated version of the data

Write queries will arrive at different nodes at different times

- Most replicated data systems will provide an eventual consistency

All nodes will have the same data

Read requests will eventually return the same value



NoSQL databases

NoSQL databases

- Non-relational storage and retrieval of data.

- Focus on

Simplicity of design

Scalability

- Horizontal scaling

Availability

Document store

- Different document-oriented databases will group documents in different ways

MongoDB:

- Databases
- Collections

Equivalent to tables in RDBMs but without a rigid schema

- A query language to interact with the DBs
-

Scaling

Scaling

- Vertical scaling (i.e., scaling up or shared-memory architectures)

The costs do not scale linearly

Limited fault tolerance

- Horizontal scaling (i.e., scaling out or shared-nothing architectures)

Costs can scale better

Better fault tolerance

Replication

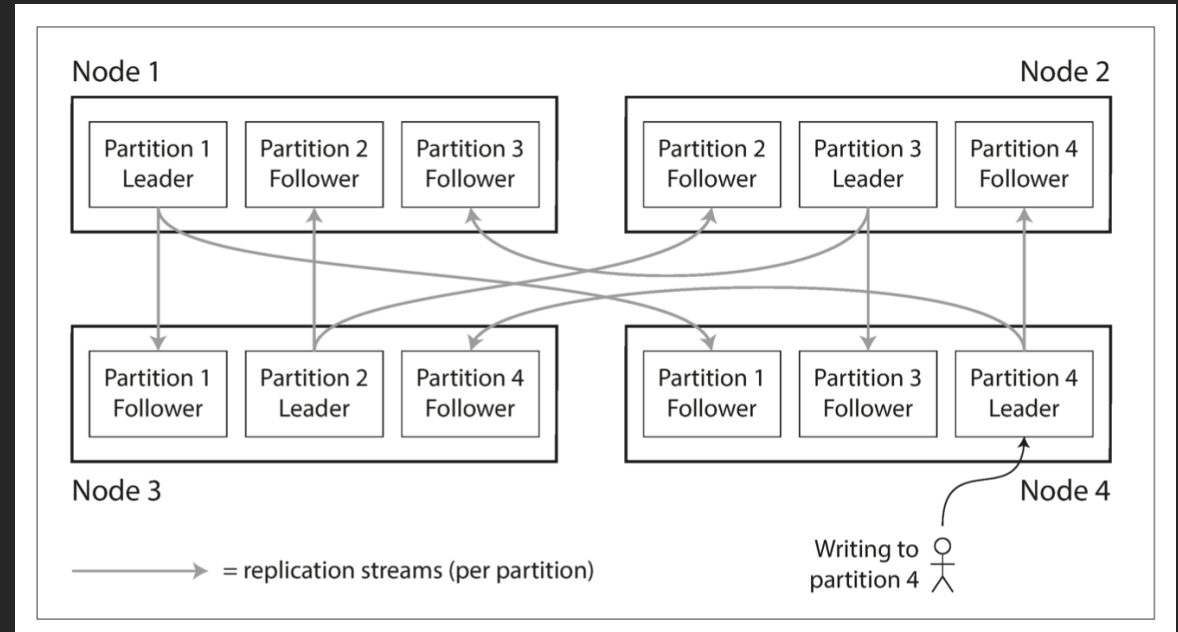
Replication

- Each node stores a copy of the data (i.e., replica)
 - Can reduce latency keeping data geographically close to the users
 - Increase availability allowing the system to continue working even if some of its parts have failed
 - Increase performance scaling out the number of machines that can serve read queries
 - Your data has to be small enough to be stored in a single machine
 - Write operations involve updating all the copies of the data
-

Partitioning

Partitioning

- Each node stores a subset of the data (i.e., partition)
- Each piece of data will belong to exactly one partition
- Focus on scalability
The query load is spread across nodes
- Often times in combination with replication



Batch processing

Batch processing

Batch processing systems (offline systems)

- A batch processing system takes a large amount of input data, runs a *job* to process it, and produces some output data.

Jobs can take from minutes to days

Batch jobs are often scheduled to run periodically.

- The primary performance measure of a batch job is usually *throughput*
-

Stream processing

Stream processing

- A lot of data sources are unbounded because it arrives gradually over time:

E.g., Social media users produced data yesterday and today, and they will continue to produce more data

- Batch processors could process data of fixed durations (e.g., every 24h)

We would always be looking at the past

We could make this window smaller and smaller...

... until we forget time windows and analyse every event as they arrive

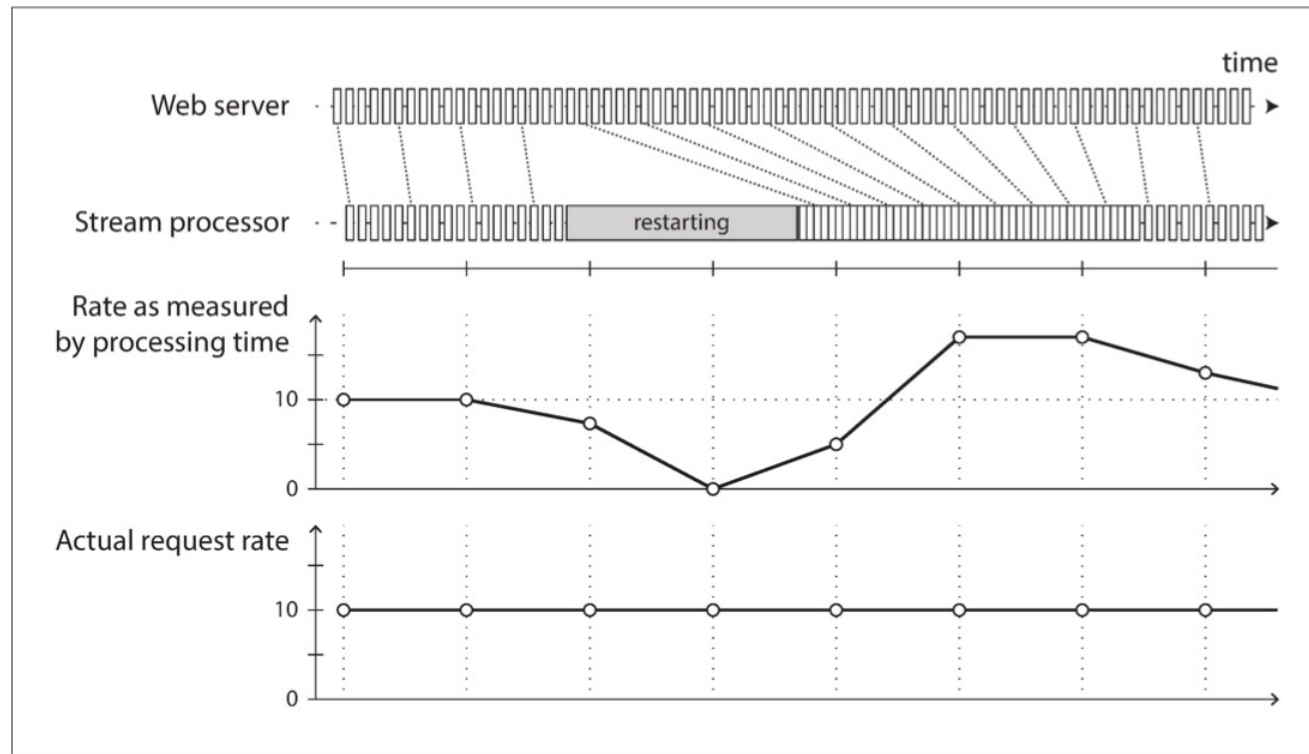
i.e., Stream Processing

Event streams

- In the stream world, data records are called events.
a small, self-contained, immutable object containing details of something that happened at some point in time.
 - Usually contains a timestamp
 - Can be encoded in many ways, as a text string, JSON, binary form ...
 - Generated by a producer
AKA publisher, or sender
 - Processed by multiple consumers
AKA Subscribers or recipients
-

Event time vs processing time

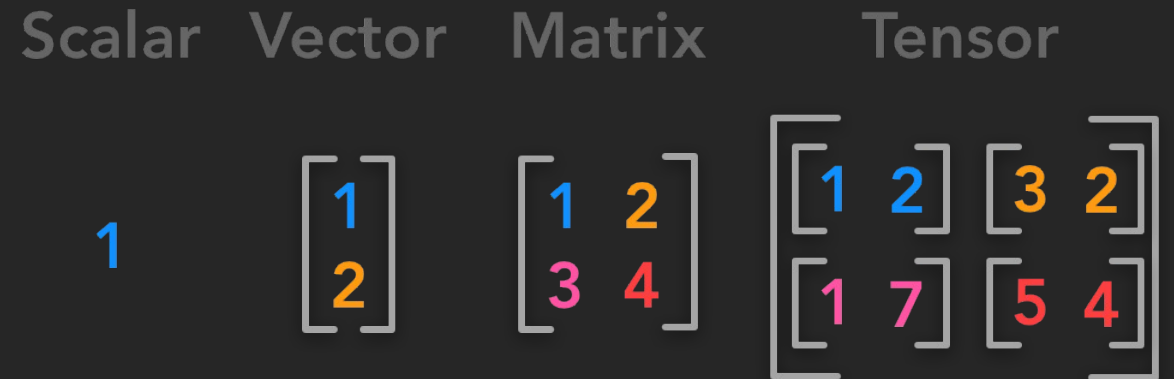
- Confusing event time and processing time leads to bad data



Tensors

Introduction

- Data is frequently represented as tensors
 - *0-order: scalar*
 - *1-order: vector*
 - *2-order: matrix*
 - *3-order: tensor*



Introduction

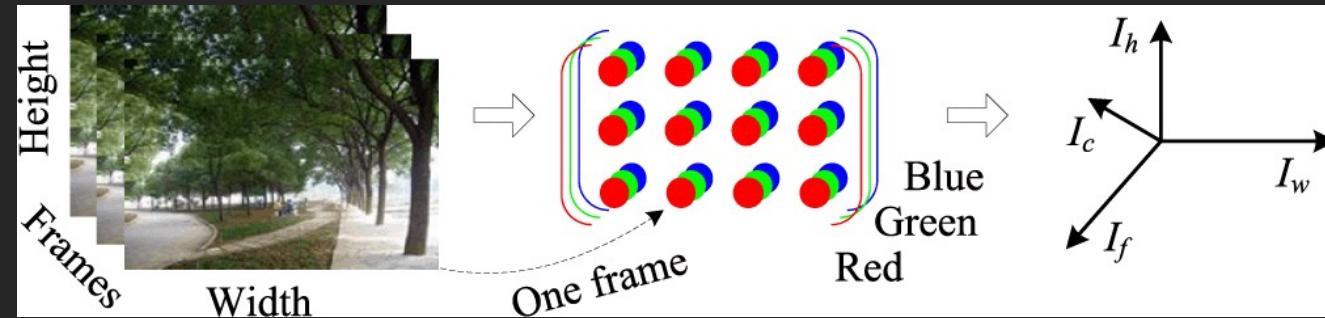
- Data is frequently represented as tensors

Vector data: features

Timeseries: features $\times$ timestamps

Image: width $\times$ height $\times$ channels (e.g. RGB)

Video: frames $\times$ width $\times$ height $\times$ channels



Sparse tensors

- Real-world data is frequently very sparse

Social graph

Fault detection sensors

- Representing large matrices ($n \times n$ with $n \sim 10^6$) might be prohibitive.

64 bits values $\rightarrow$ 58GB

- When tensors are sparse, we can find other ways to represent data more efficiently
-

Sparse tensor

- A sparse representation in which we store only the non-zero values

Dictionary of keys (row, column): value

(0,0): 1.0

(0,2): 5.0

(1,1): 3.0

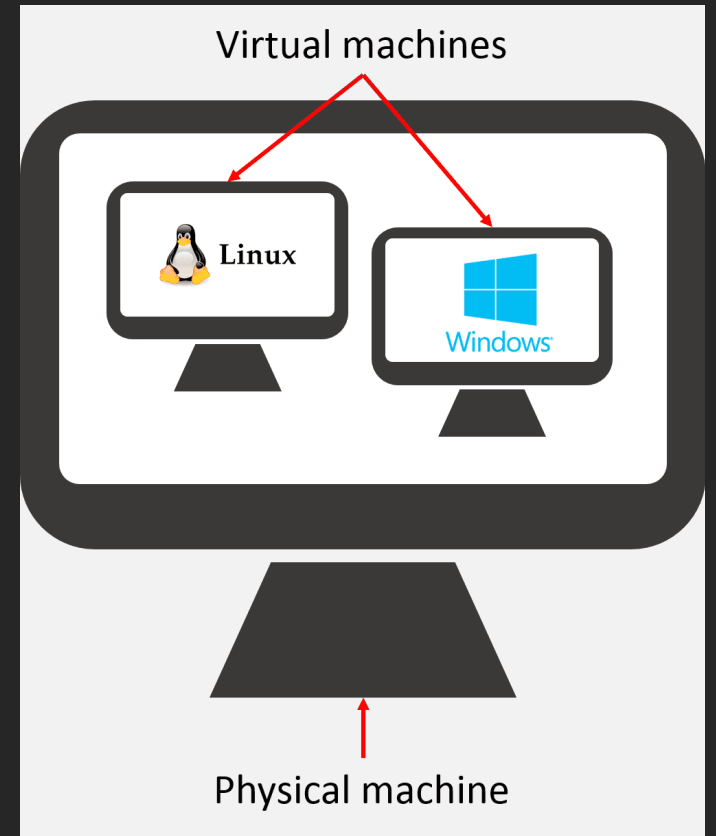
(1,5): 11.0

...

1.0	0	5.0	0	0	0	0	0
0	3.0	0	0	0	0	11.0	0
0	0	0	0	9.0	0	0	0
0	0	6.0	0	0	0	0	0
0	0	0	7.0	0	0	0	0
2.0	0	0	0	0	10.0	0	0
0	0	0	8.0	0	0	0	0
0	4.0	0	0	0	0	0	12.0

Virtualisation

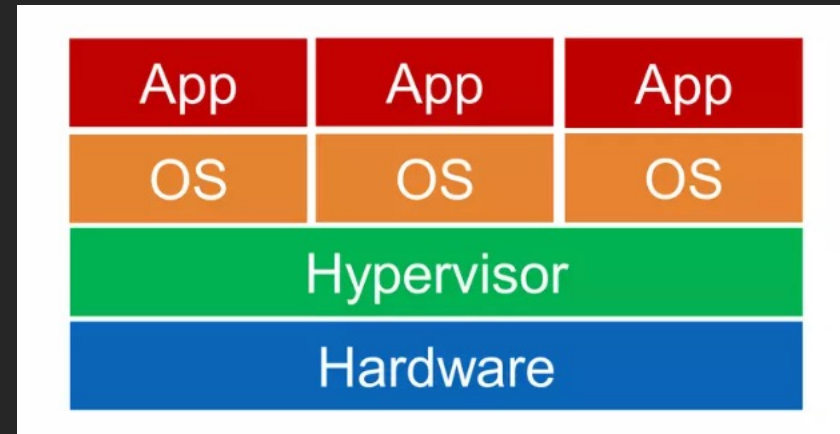
- It is the creation of a software-based *virtual machine* that acts like a real computer with all of its components (hardware, peripherals, operating system etc).
- A virtualisation software running on the physical computer (host) allowing for one or more virtual computers (guests) to run independently from one another



Drawbacks

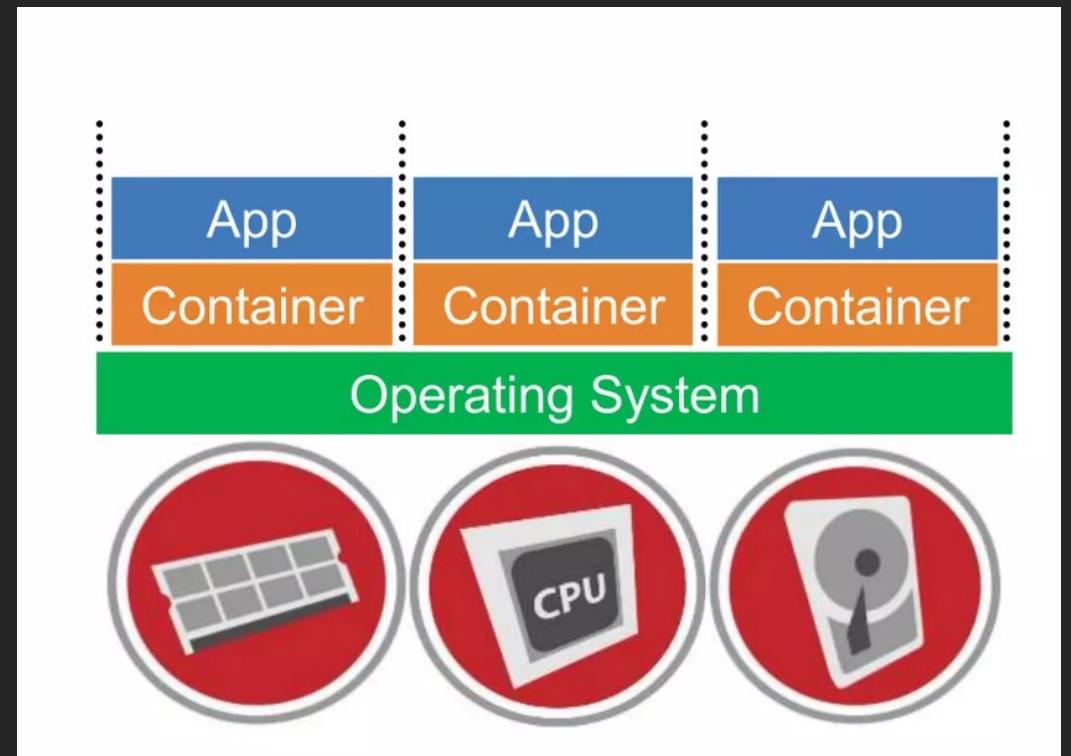
- Significant overhead
- Lots of redundancies

Each VM will have its own operating system, configuration files etc



Containerisation

- Containerisation (AKA OS-level virtualisation) enables the abstraction at the operating system level, allowing multiple applications to share binaries while remaining isolated from one another
- Containers can only use the resources that have been allocated to them



Containerisation

- A container is essentially a fully packaged and portable computing environment.
- Containers are meant to be:

portable

scalable

easier to build and deploy

easier to manage

isolated
